

GitHub develop → master PR Shows Unexpectedly Large Changes

Step-by-step troubleshooting and resolution guide for a repository where the normal workflow is feature branches → develop → master → production, but a develop → master PR shows many files that were already merged to master and are already in production.

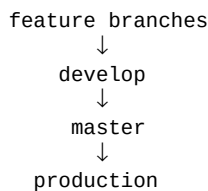
The short version

Your code was not necessarily the problem. Your Git history may be. If GitHub shows 147 files changed but a direct comparison of the current branch contents shows only 7 files changed, the branches may have a history/merge-base problem rather than 147 real new code changes.

The most important rule is: **do not immediately rebase, reset, force-push, or otherwise rewrite develop or master.** First compare the actual trees, investigate the history, and perform any reconciliation on a temporary branch.

GitHub develop → master shows unexpectedly huge changes

Assume your normal workflow is:



Step 0 — Protect yourself

Start from a clean working tree:

```
git status
```

You want: **nothing to commit, working tree clean.**

Then update remote information:

```
git fetch origin
```

Do not modify develop or master yet.

Step 1 — Check the actual content difference

```
git diff --stat origin/master origin/develop
```

Then:

```
git diff --name-status origin/master origin/develop
```

If GitHub says 147 files changed but this says 7 files changed, then you have identified the important situation: the branches have a history/merge-base problem rather than 147 files of actual code differences.

If both GitHub and the direct comparison show approximately 147 files, then it is probably a real code difference, and you should not perform the history reconciliation procedure below without investigating what those changes are.

Step 2 — Check how far the histories have diverged

```
git log --oneline --left-right origin/master...origin/develop
```

< means the commit is only in master. > means the commit is only in develop.

If you see lots of old “Merge pull request ... from .../develop” commits on the master side, that is consistent with a long-running develop → master workflow.

Step 3 — Find the common ancestor

```
git merge-base origin/master origin/develop
```

This gives you the commit where Git considers the two branches to have last shared history.

Inspect it with:

```
git show --stat <commit-hash>
```

Step 4 — Check how previous PRs were merged

Open previous develop → master PRs in GitHub and check whether they used:

- Merge pull request
- Squash and merge
- Rebase and merge

If your organization uses different merge strategies at different times, that can cause this kind of history divergence.

Step 5 — Compare the GitHub PR with the direct content difference

GitHub's PR comparison uses the branch history/merge base. The actual current files can be compared with:

```
git diff origin/master origin/develop
```

If you get GitHub: 147 files and Actual: 7 files, **do not try to manually fix 147 files**. The next step is branch-history reconciliation.

Step 6 — NEVER experiment directly on develop

```
git switch develop
git pull origin develop
git switch -c sync-master-into-develop
```

Your real develop branch remains untouched.

Step 7 — Merge master into the temporary branch

```
git merge origin/master
```

Case A — Merge completes automatically: continue to Step 8.

Case B — Merge conflicts: **stop and do not blindly choose ours/theirs**.

```
git status
```

Remember: HEAD / ours = develop; theirs = master. Resolve each conflict according to which code should exist on develop.

Case C — You realize the branches aren't actually supposed to be reconciled:

```
git merge --abort
```

Your real develop branch remains untouched.

Step 8 — Resolve conflicts based on business intent

Do not resolve Git conflicts based solely on “ours” or “theirs.” Resolve them based on which code should exist on develop.

If develop intentionally deleted something:

```
develop: DELETE
master:  MODIFY
```

and the deletion is legitimate:

```
git rm <file>
```

If develop renamed a class, keep the renamed version and update references appropriately.

If both sides modified the same code, actually inspect the differences and combine the appropriate changes.

Step 9 — Verify the resulting content BEFORE committing

```
git diff --stat origin/master
git diff --name-status origin/master
```

This answers: “What would the resulting branch actually contain that is different from production?” If the answer is approximately what you expect, for example 7 files, then you are in good shape.

Step 10 — Commit the reconciliation

```
git commit -m "Merge master into develop"
```

The merge commit has both histories as parents.

Step 11 — Push the temporary branch

```
git push -u origin sync-master-into-develop
sync-master-into-develop
↓
develop
```

This is a synchronization PR. It is not your production PR.

Step 12 — Inspect the synchronization PR carefully

Before merging it, look at the Files changed tab and the Commits tab. Ask whether the PR is introducing unexpected application changes. Ideally, it should contain little or no unexpected code change because the merge result is essentially the existing develop tree.

If GitHub suddenly shows 147 files changed, or some other unexpectedly large change in this synchronization PR, stop and investigate.

Step 13 — Merge synchronization PR into develop

```
sync-master-into-develop
↓
develop
```

Merge it using your team's normal GitHub process.

Step 14 — Update your local develop

```
git switch develop
git pull origin develop
git diff --stat origin/master origin/develop
```

This should represent the actual current code difference.

Step 15 — Now create the REAL production PR

```
develop
↓
PR
↓
master
```

On GitHub create a PR with base: master and compare: develop. At this point Git knows that the current master history has already been incorporated into develop, so the PR should focus on the actual pending changes.

Your specific case as an example

```
master
■
■■■ many previous develop → master merges
■
■■■ fa023acc
    Java/Spring upgrade

develop
■
■■■ 5538aa36 Java/Spring upgrade
■■■ 6b7ba66f CIREf cleanup
■■■ 817eb2bb mailing fix
■■■ 9ff8938f mock rename
■■■ c1d60dd8
```

GitHub saw 147 files, but the direct comparison showed 7 files. A temporary sync-master-into-develop branch was created, master was merged into it, the CIREf cleanup was preserved, and the resulting tree still differed from master by only 7 files.

The resulting merge commit was b2443550, and the verification showed 7 files changed, 4 insertions, and 520 deletions when comparing the resulting tree with origin/master.

Commands to memorize

```
git status
git fetch origin

git diff --stat origin/master origin/develop

git log --oneline --left-right origin/master...origin/develop

git merge-base origin/master origin/develop

git switch develop
git pull origin develop
git switch -c sync-master-into-develop
git merge origin/master

# After resolving conflicts:
git diff --stat origin/master
git diff --name-status origin/master

git commit -m "Merge master into develop"
git push -u origin sync-master-into-develop
```

Golden rule

First compare the actual trees. Then investigate history. Never change branch history just because GitHub shows a large PR.

For production repositories, always do the first reconciliation on a temporary branch, exactly as described in this guide.